Paper: P0805R0
Audience: Library Evolution, Library
Title: Comparing Containers
Author: Marshall Clow mclow.lists@gmail.com

Rationale

When I was attempting to implement P0122R5, I became frustrated with the inability to compare spans of different lengths, and sometimes spans of the same lengths as well. I started thinking about this in general, and realized that our comparison conventions for containers are limited, but for no good reason.

In C++17, `std::optional` has a nice set of comparison operators.

- An `optional<T>` can be compared to a `T`.
- An `optional<T>` can be compared to a `U`.
- An `optional<T>` can be compared to an `optional<T>`.
- An `optional<T>` can be compared to an `optional<U>`.
- An `optional<T>` can be compared to a `nullopt_t`.

But an `array<int, 5>` cannot be compared to an `array<int, 6>`. Nor can an `array<int, 5>` cannot be compared to an `array<long, 5>`. This seems like an artificial limitation to me - these comparisons "make sense", we just don't implement them. We *can* compare a `vector<int>` that contains five elements to a `vector<int>` that contains six elements, but not a `vector<int>` to a `vector<long>`, no matter what size. And if the allocators are different, that won't work either.

As people write more and more generic code, these arbitrary limitations will cause increasing friction.

In the header `<algorithm>` we have `lexicographical_compare`. This is a very general algorithm - it compares two sequences, which may be of differing lengths, and/or different underlying types, and returns whether one is "less than" the other. But that generality is not reflected in the containers, even though their comparisons are defined in terms of `lexicographical_compare` (Table 85).

The same logic applies to `std::tuple`. The comparison operators for `tuple` require that both tuples be the same size ([tuple.rel]/1) but (surprisingly) not that they contain the same types.

Suggested changes

The changes here are just for `array` and `tuple`; if there is interest, I can provide wording for the rest of the sequence containers.

These wording changes are relative to N4687.

In [array.syn], add the following functions:

```
template <class T1, class T2, size_t N1, size_t N2>
  bool operator==(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
  bool operator!=(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
  bool operator<(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
  bool operator>(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
  bool operator<=(const array<T1,N1>& x, const array<T2,N2>& y);
```

```
template <class T1, class T2, size_t N1, size_t N2>
  bool operator>=(const array<T1,N1>& x, const array<T2,N2>& y);
```

In [tuple.rel]/1,

- replace: “`i < sizeof...(TTypes)`” with “`i < min(sizeof...(TTypes), sizeof...(UTypes))`”
- and delete: “`sizeof...(TTypes) == sizeof...(UTypes).`”

In [tuple.rel]/2,

- add: “`false` if `sizeof...(TTypes) != sizeof...(UTypes)`, otherwise ” after *Returns:*

In [tuple.rel]/4,

- replace: “`i < sizeof...(TTypes)`” with “`i < min(sizeof...(TTypes), sizeof...(UTypes))`”
- and delete: “`sizeof...(TTypes) == sizeof...(UTypes).`”

In [tuple.rel]/4,

- add: “A zero-length tuple is lexicographically less than any non-zero-length tuple.” after “lexicographical comparison between `t` and `u`.”

Implementation status

I have implemented the changes for both `tuple` and `array`.

Future Directions

- Provide similar functionality for `vector`, `list`, `deque`, and `forward_list`.
- Provide compatibility with P0515 - Consistent Comparisons.
- Research and decide if similar functionality should be provided for the associative containers. P0809R0 - “Comparing Unordered Containers” (no link yet) may influence this direction.